Ders 24 Genişletilmiş Notu: Tekrar Dersi: Sütun Silme, Kümelerde Bağıntı Özellikleri ve Polinomlar

Ders 24 Genişletilmiş Notu: Tekrar Dersi: Sütun Silme, Kümelerde Bağıntı Özellikleri ve Polinomlar

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Matristen Sütun Silme
- 2. Kümelerde İkili Bağıntılar ve Simetri Özelliği
- 3. Simetriyi Bağıntı Matrisiyle Kontrol Etmek
- 4. Yansıma Özelliği ve Matrisin Dezavantajı
- 5. Simetriyi Matris Kullanmadan Kontrol Etmek
- 6. Kümelerle İlgili Başka Problemler
- 7. Polinomlar: Kök Bulma ve Denklemi Yazdırma
- 8. Soru-Cevap: C Programlama ve İşletim Sistemleri
- Dersin Toparlaması

1. Matristen Sütun Silme

Hoca bu tekrar dersine, ara sınavda sorduğu sütun silme sorusunu çözerek başladı. Soruyu günlük bir işe bağladı: Excel'de aradan bir sütun ya da satır silmeyi sık sık yaparız ve burada arka planda genellikle bir kaydırma işlemi yapılır.

1.1 Problem ve Örnek

Dışarıdan matrisin boyutları (N satır, M sütun), matrisin kendisi ve silinecek sütunun numarası (X) alınır. Sütun silinince sağındaki sütunlar birer sola kayar, boşalan son sütun sıfırlanır.

Hocanın örneğinde matrisin değerleri 1'den 12'ye kadardır ve ikinci sütun silinmek istenir. Beklenen sonuç, hocanın okuduğu sırayla, 1, 2, 3, 7, 8, 9, 10, 11, 12 ve 0'dır.

> **Ders sonrası not:** Altyazıda matrisin dizilişi söylenmiyor. Hoca sonraki uç durumda M = 4 aldığı için matris büyük olasılıkla 3 satır, 4 sütunludur ve değerler sütun sütun okunmuştur. Python ile denendi: Bu dizilişte ikinci sütun silinince okunan sonuç aynen çıkıyor. Değerler satır satır dizilmiş olsaydı aynı sonuç çıkmazdı. Hocanın "ve 0" dediği yer de büyük olasılıkla son sütunun üç sıfırıdır.

1.2 Algoritma

Akış diyagramının adımları şöyledir (ekrandan, 00:09:50, kısa nottan):

1. N, M, `matris[N][M]` ve silinecek sütun X okunur.
2. `J ← X+1 || 1 || M` döngüsü açılır. Her J için `I ← 1 || 1 || N` döngüsünde `matris[I][J-1] = matris[I][J]` yapılır.
3. `I ← 1 || 1 || N` döngüsünde `matris[I][M] = 0` yapılır.
4. Dur.

İkinci adımın fikri, silinecek sütunun bir sağındaki sütunu silinecek sütunun üzerine taşımak, sonra bir sonrakini onun üzerine taşımak ve böylece M'e kadar gitmektir. Bu işlem her sütunda bütün satırlar (bütün I'lar) için yapılır. Kaydırma bittiğinde son sütundaki değerler yerinde durmaya devam eder, yani bir kopyası oluşmuştur. Bu yüzden son adımda son sütun sıfırlanır.

1.3 Örnek Üzerinde Adım Adım (X = 2)

Verilen matris (N = 3, M = 4):

$$\begin{pmatrix} 1 & 4 & 7 & 10 \\ 2 & 5 & 8 & 11 \\ 3 & 6 & 9 & 12 \end{pmatrix}$$

$J = 3$: Üçüncü sütun ikinci sütunun üzerine yazılır (ara adım eklendi).

$$\begin{pmatrix} 1 & 7 & 7 & 10 \\ 2 & 8 & 8 & 11 \\ 3 & 9 & 9 & 12 \end{pmatrix}$$

$J = 4$: Dördüncü sütun üçüncü sütunun üzerine yazılır. Hocanın dediği gibi, bu noktada son sütunda hâlâ 10, 11, 12 yazmaktadır (ara adım eklendi).

$$\begin{pmatrix} 1 & 7 & 10 & 10 \\ 2 & 8 & 11 & 11 \\ 3 & 9 & 12 & 12 \end{pmatrix}$$

Son sütun sıfırlanır ve sonuç elde edilir:

$$\begin{pmatrix} 1 & 7 & 10 & 0 \\ 2 & 8 & 11 & 0 \\ 3 & 9 & 12 & 0 \end{pmatrix}$$

Sütun sütun okununca 1, 2, 3, 7, 8, 9, 10, 11, 12, 0, 0, 0 çıkar.

> **Ders sonrası not:** Hoca bu kısımda ve hemen ardından "son satırı sıfırlamak", "silinmek istenen satır" dedi (00:06:26, 00:06:59). Akış diyagramı son sütunu sıfırladığı ve soru sütun silme sorusu olduğu için hoca büyük olasılıkla sütunu kastetti.

1.4 Farklı Çözüm: Atamayı Ters Yönden Yazmak

Hoca, kağıtlarda farklı çözümler de gördüğünü söyledi. Bazı öğrenciler atamayı `matris[I][J] = matris[I][J+1]` biçiminde yazmıştır, yani J+1'i J'ye atamışlardır. Hoca bunun da olabileceğini belirtti. Kendi çözümünde J'yi X+1'den başlatıp bir soldaki göze atama yapmıştır. İki yazım aynı kaydırmayı yapar.

1.5 Uç Durum: Son Sütunu Silmek

Soru: Silinmek istenen sütun arada değil de en sonda olsaydı algoritma hataya düşer miydi?

Sınıftan farklı yorumlar geldi. Bir öğrenci hata olmayacağını, yalnızca bir döngüye girilmeyeceğini söyledi, ama hangi döngü olduğunda karışıklık oldu. Başka bir öğrenci ilk döngüye girilmeyeceğini, sadece son döngüye girileceğini söyledi.

Cevap: Hoca $X = 4$ için algoritmayı adım adım izledi:

- J, $X + 1 = 5$'ten başlar. M de 4 olduğu için döngü koşulu $5 \leq 4$ olur.
- Bu koşul yanlıştır, dolayısıyla kaydırma döngüsüne hiç girilmez ve matriste hiçbir değişiklik yapılmaz.
- İkinci döngüde I, 1'den N'e kadar gider ve `matris[I][M]` gözleri, yani silinmek istenen son sütunun değerleri sıfırlanır.

Yani algoritma son sütun için de doğru çalışır. Python ile aynı örnekte denendiğinde sonuç sütun sütun 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 0, 0 çıkıyor.

Hoca bu soruyu ısrarla sormasının sebebini şöyle açıkladı: En son sütunu silmek, en baştaki sütunu silmek ve ortadaki bir sütunu silmek birbirinden farklı durumlardır (case). Program çıktısı gösterilirken "farklı durumlar" denildiğinde kastedilen de budur.

> **Hocanın vurgusu:** Uç noktaları, yani en baştaki, ortadaki ve en sondaki sütunu silmek gibi durumları ayrı ayrı denemek gerekir. Bunların her biri farklı bir durumdur.

2. Kümelerde İkili Bağıntılar ve Simetri Özelliği

Hoca ikinci olarak kümeler üzerine bir örnek soruya geçti. Bir kümenin elemanları için ikililer yazılır. Hoca bu ikilileri yol gibi de düşündürdü: $(1, 2)$ verilmişse 1'den 2'ye yol var demektir.

Hoca, sınıfın konuya yabancı kaldığını görünce şaşırdığını söyledi. Bu konuların bağıntı ve fonksiyonlar kapsamında daha önce görülmüş olması gerektiğini düşünüyordu. Sonra konuyu şöyle toparladı: Verilen bir kümenin elemanları arasındaki bağıntılardan yola çıkarak bağıntının simetri, yansıma, geçişkenlik, ters simetri gibi özellikleri taşıyıp taşımadığına bakarız. Bu kontrolü de verilen ikili girdiler üzerinden yapmak isteriz.

> **Ek bilgi (derste anlatılmadı):** Hoca ters simetrinin tanımını vermedi, geçişkenliği de yalnızca bir örnekle andı. Ters simetri büyük olasılıkla şu anlamda kullanıldı: (a, b) ve (b, a) birlikte bulunuyorsa $a = b$ olmalıdır. Geçişkenliğin genel biçimi de büyük olasılıkla şudur: (a, b) ve (b, c) varsa (a, c) de bulunmalıdır.

2.1 Simetri Nedir?

Bir kümede (a, b) ikilisi varken (b, a) da varsa simetrik özellikten söz ederiz. Hocanın ifadesiyle, bir elemandan diğer elemana bağlantı varsa diğer elemandan da öbür elemana bir bağlantı olması gerekir.

Hoca önce kısa bir başlangıç örneği verdi: Küme $\{1, 2, 3, 5, 7\}$, ikililer $(1, 2), (2, 1), (3, 3)$. Sonra öğrencinin sorusu üzerine örneği genişletti.

Soru: Küme içinde yazınca baştan başlayarak ikilileri ardışık şekilde mi karşılaştıracağız?

Cevap: Hayır. İkililerin yerleri önemli değildir ve her ikilinin bir karşılığı olmalıdır. İkililer karışık sırada verilmiş olabilir. Dersin geri kalanında kullanılan bağıntı şudur:

$$(1, 2),\ (3, 5),\ (2, 1),\ (3, 7),\ (7, 3),\ (5, 3),\ (1, 1)$$

Burada 1'den 2'ye ve 2'den 1'e, 3'ten 5'e ve 5'ten 3'e, 3'ten 7'ye ve 7'den 3'e yol vardır.

Problemin varsayımları da açıktır: Size N tane ikili bağıntı verilir, kümenin elemanlarını ayrıca almanıza gerek yoktur. Kümenin elemanlarının dışında bir bağıntı verilme ihtimali yoktur, yani bağıntıların hatalı verilmediğini kabul ederiz. Görev, bu bağıntının simetrik özellik taşıyıp taşımadığını kontrol eden bir program yazmaktır.

2.2 $(1, 1)$ Simetriyi Bozar mı?

Bir öğrenci, listedeki her ikilinin ilk elemanını alıp diğer ikililerin ikinci elemanlarıyla karşılaştırmayı önerdi: 1'i alıp 5, 1, 7 ile karşılaştırmak gibi. Hoca bunu izlerken listeye $(1, 1)$ ekleyip bağıntının hâlâ simetrik olup olmadığını sordu.

Soru: Bağıntıya $(1, 1)$ eklenirse simetri korunur mu?

Öğrencinin ilk tepkisi, $(1, 1)$'in kendisine eşleşen başka bir eleman olmadığı için simetriyi taşımayacağı yönündeydi.

Cevap: $(1, 1)$ simetriyi bozmaz, çünkü karşılığı yine $(1, 1)$'dir. $(1, 2)$'nin karşılığında $(2, 1)$ aranır, $(1, 1)$'in karşılığında başka bir şey aranmaz. Bu yüzden bu küme simetri özelliğini taşır.

Bu örnek, önerilen algoritmanın eksiğini de gösterdi. Yalnızca ilk elemanı arayıp her eşleşmede karşılığa bakarsak şöyle olur: 1'i aldık, $(2, 1)$'i bulduk, karşılığında 2 var, sorun yok. Sonra

$(1,1)$'de yine 1'i bulduk, karşılığında 2 yok. O zaman "simetri yok" derdik ki bu hatalı olurdu. Python ile denendi: Her eşleşmede karşılığı arayan bu saf algoritma, simetrik olan bağıntıyı gerçekten simetrik değil buluyor.

Bu sorunu çözmek için iki öneri daha geldi:

- **Sıralamak:** Bir öğrenci önce sıralayıp sonra karşılaştırmayı önerdi. Hocanın cevabı, sıralamaya gerek olmadığı ve sıralamanın çok maliyetli bir iş olduğu oldu.
- **Bir `if` eklemek:** Başka bir öğrenci, "simetrik değil" kararı verilen yere bir `if` koyup ikilinin kendisini gösterip göstermediğine bakmayı önerdi. Hoca bunun da bir bakış açısı olabileceğini söyledi, ama daha kolay bir yol önerdi: matris kullanmak.

  Hocanın vurgusu: Sıralamak çok maliyetli bir iştir. Bu problemde sıralamaya gerek yoktur.

3. Simetriyi Bağıntı Matrisiyle Kontrol Etmek

3.1 Matrisi Doldurmak

Hoca matrisle nasıl çözüleceğini sordu ve bir öğrencinin önerisiyle matrisi doldurdu. Her (a,b) bağıntısı için a. satır, b. sütundaki göze 1 konur, geri kalan gözlerin 0 olduğu kabul edilir. Kümenin en büyük elemanı 7 olduğu için örnekte 7'ye 7'lik bir matris kullanıldı. Örneğin bağıntısı için doldurulan matris şudur (ara adım eklendi):

$$\begin{pmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{pmatrix}$$

3.2 Transpozla Karşılaştırmak

Öğrencinin devam eden önerisi, matrisin transpozunun kendisine eşit olup olmadığına bakmaktı. Bunun için iki döngü ile bütün (i,j) gözleri taranır ve `matris[i][j] = matris[j][i]` mi diye bakılır. Python ile denendi: Yukarıdaki matris transpozuna eşittir.

Hoca iki soru daha sordu ve bu çözümü adım adım iyileştirdi.

3.3 Hangi Döngü: for mu while mı?

Soru: Bu tarama için hangi döngü tipi kullanılır? Adım sayısı belli mi?

Öğrencinin ilk cevabı `for` oldu.

Cevap: Adım sayısı aslında belli değildir. Çünkü transpoz kontrolü sırasında `A[i][j]` ile `A[j][i]` eşit değilse döngüyü kesmek gerekir. İlk karşılaşılan eşitsizlikte çıkma ihtimali olduğu için `while` döngüsü kullanılmalıdır.

> **Hocanın vurgusu:** İlk eşitsizlikte döngüden çıkılacağı için adım sayısı belli değildir. Bu durumda `while` kullanılır.

3.4 Bütün Matrisi Gezmemek

Bir öğrenci, bütün matrisi taramak yerine yalnızca değeri 1 olan gözleri, yani verilen ikilileri gezmeyi önerdi. Bu durumda kaç ikili olduğu bilindiği için `for` da kullanılabilir. Hoca fikri sınıfa açıkladı: Elimizdeki her (i,j) ikilisi için matriste tam karşısındaki (j,i) gözünde 1 var mı diye bakılır.

Örnek üzerinde kontroller şöyledir:

- $(1,2)$ için $(2,1)$ var.
- $(3,5)$ için $(5,3)$ var.

- $(2,1)$ için $(1,2)$ var.
- $(3,7)$ için $(7,3)$ var.
- $(7,3)$ için $(3,7)$ var.
- $(5,3)$ için $(3,5)$ var.
- $(1,1)$ için yine $(1,1)$ var.

Toplamda 7 kontrol yeterlidir. Aksi halde 7'ye 7'lik matrisin 49 gözünün hepsi gezilir (ara adım eklendi).

Transpoz kontrolünün kendisinde de bütün matrisi gezmek gerekmez. Bir öğrencinin dediği gibi yarısını gezmek yeterlidir. Matrise bir köşegen (diagonal) çektiğinizi düşünün. Köşegenin yalnız bir tarafını gezip her (i,j) gözünün karşılığı olan (j,i) gözüne eşit olup olmadığına bakmak, transpozun matrise eşitliğini kontrol etmekle aynı şeydir.

> **Hocanın vurgusu:** Transpoz kontrolünde bütün matrisi gezmeyin. Köşegenin bir tarafı yeterlidir. Hoca bunun unutulmamasını özellikle istedi.

4. Yansıma Özelliği ve Matrisin Dezavantajı

4.1 Yansıma Özelliği

Yansıma özelliğinde, kümedeki bütün elemanların kendisine karşılık gelen bir bağıntı olmalıdır. Yani bütün x elemanları için (x,x) bağıntıda bulunmalıdır: $(1,1),(2,2),(3,3)$ gibi.

Matris kullanıyorsak bunu kontrol etmek kolaydır. Bir öğrencinin söylediği gibi köşegen üzerindeki değerlerin 1 olup olmadığına bakılır.

> **Ders sonrası not:** Hoca köşegeni 1, 2, ..., 7 diye saydı ve "1,1'den 7,7'ye kadar hepsi varsa" dedi (00:31:01, 00:33:12). Ama örnek küme $\{1,2,3,5,7\}$ olduğundan 4 ve 6 kümenin elemanı değildir. Bu küme için büyük olasılıkla yalnız $(1,1),(2,2),(3,3),(5,5),(7,7)$ gerekir, yani köşegenin yalnızca elemanlara karşılık gelen gözlerine bakılmalıdır. Hoca büyük olasılıkla o an 1'den 7'ye kadar bütün sayıların eleman olduğu bir kümeyi düşünüyordu. Python ile denendi: Dersteki bağıntı simetriktir ama yansıma özelliği taşımaz, çünkü $(2,2),(3,3),(5,5),(7,7)$ yoktur. Hocanın çizdiği köşegeni 1 olan matris büyük olasılıkla yalnız anlatım içindir.

Soru: Yansıma için ikilileri girdi alınırken karşılaştırabilir miyiz? İki değer alınır, farklı bir çift gelince okuma kesilir.

Cevap: Hayır, farklı iki değerin gelmesi yansımayı bozmaz. Örneğin $(1,1),(2,2),(3,3)$ varken $(4,5)$ gelmesi sorun değildir. Elemanların hepsi için (x,x) varsa yansıma sağlanır. Bunlardan biri bile yoksa yansıma ortadan kalkar.

4.2 Matrisin Dezavantajı

Soru: Matristen faydalanmanın bir dezavantajı var mı?

Cevap: Bir öğrencinin belirttiği gibi, sayılardan biri 999 gibi büyükse matris çok fazla alan kaplar. Kümedeki elemanlar büyük sayılardan oluşuyorsa 999'a 999 gibi çok büyük bir alan açmak gerekir.

5. Simetriyi Matris Kullanmadan Kontrol Etmek

Hoca bu kez soruyu şöyle sordu: Simetri kontrolünü bir matris yardımı olmadan, bellekte ekstra alan kullanmadan yapabilir miyiz? Girdiler yine iki sütunlu bir yapıda saklanacaktır ve yalnızca bu yapı kullanılacaktır.

5.1 Algoritma

Bir öğrencinin önerisi hoca ile birlikte adım adım netleşti. Algoritmanın düz metin hali şöyledir:

1. Dış döngü her (a,b) ikilisini sırayla alır.
2. İç döngüye girmeden önce `flag = 0` yapılır.

3. İç döngü listenin en başından başlayarak, ikilinin kendisi de dahil olmak üzere (b, a)'yı arar. Bulunca `flag = 1` yapılır ve iç döngüden çıkılır.
4. İç döngüden çıkılınca `flag` 1 ise durum bozulmamıştır, sonraki ikiliye geçilir.
5. `flag` 0 ise simetri bozulmuştur, dış döngüden de çıkılır ve program sonlanır.

İki ayrıntı önemlidir:

- **Arama listenin başından başlar.** Kendisinden sonrakilere bakmak yetmez. Örneğin $(2, 1)$'in karşılığı olan $(1, 2)$, ondan önce gelir.
- **Karşılık bulununca aramadan çıkılır.** Hoca başta bu çıkışı yanlış anladığını, sonra öğrencinin ne demek istediğini anladığını söyledi. Bir defa bulmak yeterlidir. Aksi takdirde arama devam ederken $(1, 1)$ gibi başka bir eşleşme hatalı karara yol açabilir.

Örnekte iç döngülerin sonucu şöyledir: $(1, 2)$ için $(2, 1)$, $(3, 5)$ için $(5, 3)$, $(2, 1)$ için en baştan bakılınca $(1, 2)$, $(3, 7)$ için $(7, 3)$, $(7, 3)$ için $(3, 7)$, $(5, 3)$ için $(3, 5)$ bulunur. Son olarak $(1, 1)$ için yukarıdan aşağıya bakılır ve en sonunda kendisi bulunur. Bu yüzden aramanın kendisini de kapsaması gerekir.

5.2 Aykırı Bir Eleman Varsa

Soru: Simetriyi bozan bir eleman olsaydı bunu nasıl anlayacaktık?

Cevap: Hoca burada `flag` kullanımını gösterdi ve "buraya 3'e 1 koyalım" dedi. Akış şöyledir:

1. $(1, 2)$ alınır, `flag = 0` yapılır. Listede ikinci elemanı 1 olanlar aranır.
2. $(3, 1)$'de 1 bulunur. Karşılığı olan 3, 2'ye eşit değildir. Yola devam edilir.
3. $(1, 1)$'de yine 1 bulunur. Karşılığı olan 1, 2'ye eşit değildir.
4. Döngüden `flag = 0` olarak çıkılır. Dış döngüden de çıkılır ve program sonlanır.

> **Ders sonrası not:** Altyazıda "buraya 3'e 1 koyalım" (00:40:05-00:40:19) deniyor, ne yazıldığı açıkça söylenmiyor. Hocanın izlemesi $(1, 2)$'de `flag = 0` ile bittiğine göre $(3, 1)$ büyük olasılıkla $(2, 1)$'in yerine yazıldı, çünkü $(2, 1)$ listede kalsaydı $(1, 2)$ karşılığını bulurdu. Python ile iki durum da denendi: $(2, 1)$ yerine $(3, 1)$ yazılınca $(1, 2)$ ve $(3, 1)$ karşılıksız kalır, $(3, 1)$ yalnızca eklenince sadece $(3, 1)$ karşılıksız kalır. İki durumda da bağıntı simetrik değildir.

Soru: İç döngüye girmeden hemen önce `flag`'i her seferinde sıfır mı yapıyoruz?

Cevap: Evet.

> **Ders sonrası not:** Hocanın akışı C'de büyük olasılıkla aşağıdaki gibi yazılır. Hoca bu kodu derste yazmadı. Kod bir `main` içinde `gcc -Wall` ile derlendi. Dersteki bağıntı için 1, $(2, 1)$ yerine $(3, 1)$ yazılınca 0 döndürdü.

```c
int simetrik_mi(int bag[][2], int K)
{
    int i = 0, j, flag = 1;

    while (i < K && flag == 1) {
        flag = 0;                /* her bagintidan once 0 */
        j = 0;
        while (j < K && flag == 0) {
            if (bag[j][1] == bag[i][0] && bag[j][0] == bag[i][1])
                flag = 1;        /* (b,a) bulundu */
            j++;
        }
        i++;
    }
    return flag;
}
```

5.3 Öğrencilerden Gelen Diğer Sorular

Soru: Değerleri tek tek aramak yerine ikiliyi yer değiştirip $(2, 1)$'i bütün satırlarda arasak, yani tek döngüyle çözsek daha kolay olmaz mı?

Cevap: Aynı kapıya çıkar. $(1, 2)$'yi alıp geri kalanlarda $(2, 1)$'i aradığınızda birinci tur biter. Dış döngüde ikinci tura başlayıp $(3, 5)$'i almanız gerekir. Yani yine iki döngü vardır.

Soru: $(1, 2)$ ikilisinden iki tane verilmiş olsaydı?

Cevap: Böyle bir ihtimal yoktur. Bu bir küme bağıntısıdır ve kümelerde aynı bağıntıdan iki tane olmaz. Bağıntıların hatalı verilmediğini kabul ediyoruz.

6. Kümelerle İlgili Başka Problemler

Hoca sınıfa kümelerle ilgili başka hangi problemlerin sorulabileceğini sordu. Gelen cevaplar şunlardı:

- **Geçişkenlik:** $(1, 2)$ ve $(2, 3)$ varken $(1, 3)$'ün de bulunması durumu. Bu da bağıntılar arasındaki bir özelliktir.
- **Kesişim:** İki kümenin kesişimi var mı, kesişim kümesinde hangi elemanlar var?
- **Birleşim:** İki kümenin birleşim kümesi.
- **Fark:** X kümesinde olup Y kümesinde olmayan elemanlar ya da tersi.
- **Alt küme:** Bir küme diğerinin alt kümesi mi? Bir kümedeki elemanların hepsi diğerinde bulunmalıdır. Bir tanesi bile yoksa alt küme değildir.
- **Kapsama:** Bir küme diğerini kapsıyor mu? Bu, alt küme sorusunun tersidir.

Hoca bu şekilde en azından 3-4 soru üretildiğini söyledi. Matematikle ilgili birçok problemi bilgisayar üzerinde çözme imkanımız vardır ve istenen özelliklerin sağlanıp sağlanmadığını hızlıca test edebiliriz.

7. Polinomlar: Kök Bulma ve Denklemi Yazdırma

7.1 Kök Bulma

Matematikte bilgisayarla çözülebilecek başka bir problem de kök bulmadır: Bir denklemin kökü var mı, varsa ne ve kaç tane?

Bir öğrenci, ikinci dereceden bir polinomda delta (diskriminant) hesaplandığını söyledi: $\Delta = b^2 - 4ac$. $\Delta \geq 0$ ise kök vardır, $\Delta < 0$ ise yoktur. Derste köklerin formülü tam hatırlanamadı. Hoca da yıllardır kullanmadığı için formülü ezbere hatırlamadığını söyledi. Önemli olan, bu tür soruların formül bilgisi kullanılarak bilgisayarda cevaplanabilmesidir.

> **Ders sonrası not:** Bir öğrenci paydanın $2a$ olduğunu hatırladığını söyledi. Altyazıda formülün kalanı geçmiyor. Formül büyük olasılıkla şudur: $x_{1,2} = \dfrac{-b \pm \sqrt{\Delta}}{2a}$. Derste "kökü vardır" denen $\Delta \geq 0$ durumu da büyük olasılıkla gerçel kökleri kasteder.

7.2 Katsayılardan Denklemi Yazmak

Hoca, bir denklemin katsayıları ve polinomun derecesi verilirse bir x değeri için y'nin kaç olacağının söylenebileceğini, bu hesabı bilgisayarda yapabilmek için bu bilgilerin girilmesi gerektiğini belirtti.

Örnek: Dördüncü dereceden bir fonksiyonun katsayıları 1, 5, 0, 2, 7 olsun. Denklem şöyledir:

$$x^4 + 5x^3 + 2x + 7$$

0 katsayısı x^2 terimine karşılık gelir, bu yüzden x^2 yazılmaz. Hoca burada bir eksiklik olduğunu söyleyip sınıfa sordu. Gelen cevaplar x^2'nin belirtilmemesi, yazımın açık olmaması, katsayıların sıralı olması ve "eşittir sıfır" oldu. Hoca bunların hiçbirini kabul etmedi. Kastettiği şey, katsayılar arasında eksi değerlerin de olabileceğiydi. Örneğin 5 yerine -5 verilseydi denklem şöyle olurdu:

$$x^4 - 5x^3 + 2x + 7$$

Yani artı ve eksi değerler verilerek terimin işareti de belirlenir ve denklem bu şekilde ekrana yazdırılabilir.

7.3 Girdinin Nasıl Verildiği Önemlidir

Hoca bu noktada sınıfa, yalnızca katsayı bilgisi verilse denklemi bu biçimde yazdırıp yazdıramayacağımızı sordu.

İki öğrenci görüş bildirdi. Birincisi, en büyük derece bilinmediği için (x^5 mi, x^4 mü) yazılamayacağını söyledi. Hoca bunun bir bakış açısı olduğunu belirtti. İkincisi, sıfır olan terimler için de 0 verildiyse yazılabileceğini, ama aksi halde en küçük dereceli terimin bilinemeyeceğini söyledi.

Hocanın cevabı, $ax^n + bx^{n-1} + \ldots$ biçiminde başlandığı düşünülürse bütün üsler için katsayılar verildiğinde sorun olmadığıdır. x^4'ün katsayısı 1, x^3'ünki -5, x^2'ninki 0 olur. Olmayan terimler zaten 0 olarak verileceği için dizi verildiğinde herhangi bir problem yoktur.

> **Hocanın vurgusu:** Size verilen girdi ve girdiyle ilgili bilgi çok önemlidir.

7.4 Katsayılar String Olarak Verilirse

Hoca problemi zorlaştırdı. Dışarıdan iki string verilsin: `"1507"` ve `"-1507"`. Bunlardan denklemi yazabilir miyiz? Bir öğrenci pozitif olanın kolay, negatif olanın ise o kadar açık olmadığını söyledi. Hoca bu tür farklı durumlar için algoritmik bakış açısı geliştirmek gerektiğini belirtti.

`"-1507"` içinde şu karakterler vardır: `-`, `1`, `5`, `0`, `7`. Bir öğrencinin önerisi hoca ile netleşti:

1. String karakter karakter gezilir. Hoca işaret olarak yalnızca eksiyi düşünmeyi, artıyı unutmayı söyledi.
2. Karakter `-` ise sonraki karakter alınır ve negatif değer olarak kullanılır.
3. Karakter `-` değilse doğrudan alınır.
4. Karakterin rakamsal değerini bulmak için `'0'` karakterine göre farkı alınır.

`-1507` için katsayılar $-1, 5, 0, 7$ olur. Hocanın yazdığı denklem de şudur (x'in katsayısı 0 olduğu için x terimi yoktur):

$$-x^3 + 5x^2 + 7$$

> **Hocanın vurgusu:** String'teki değerlerle çalışırken karakterin ASCII karşılığını değil rakamsal karşılığını bulmamız gerekir. Bunun için `'0'` karakterine göre fark alınır. Hoca bunun unutulmamasını istedi.
>
> **Ders sonrası not:** Hoca "sıfırdan çıkartırız" dedi (00:54:18). Kastettiği büyük olasılıkla karakterin kendisinden `'0'` çıkarmaktır: `c - '0'`. Ters yönde yapılsa işaret ters çıkar. Python ile: `'5'` karakterinin ASCII değeri 53, `'0'` karakterininki 48'dir. $53 - 48 = 5$ çıkar, $48 - 53 = -5$ çıkar.

Soru: Bu stringde eksi her rakamdan önce mi konulur? Yani eksi yalnız başta mı olabilir? Çünkü diğer katsayıların eksi olup olmadığını bilemiyoruz.

Cevap: Hoca önce `-1` durumunu denklemle gösterdi: $-x^3 + 5x^2 + 7$. Öğrenci soruyu, $-1, -5, 0, 7$ gibi bir durumun nasıl gösterileceği şeklinde netleştirdi.

8. Soru-Cevap: C Programlama ve İşletim Sistemleri

Dersin kalan kısmında öğrenciler işlenen konularla ilgili sorularını sordu.

8.1 Karakter Sabiti mi, ASCII Değeri mi?

Soru: Kodda 65 yerine tırnak içinde `'A'` yazsak bir performans farkı olur mu?

Cevap: Hiçbir farkı yoktur.

8.2 C Öğrenmek İçin Kaynak

Soru: C kodunu yazarken zorlanıyorum. Sözdizimini, verilerin nasıl girilip işleneceğini tam bilmiyorum. Önerdiğiniz bir kitap var mı?

Cevap: Dersin izlencesinde yazan kitap kullanılabilir. Hoca ayrıca sohbetten bir adres paylaştı ve İngilizce bir kaynak olarak **Software Engineering in C** kitabını önerdi.

Hoca bu dersin amacının C öğretmek olmadığını hatırlattı. Çizilen akış diyagramlarını koda dönüştürmek için gereken C bilgisi bir iki derste verilmiş, sonra üzerinde durulmamıştır. Kendinizi bu konuda ne kadar geliştirirseniz sonraki dönem ve ikinci sınıfta o kadar rahat edersiniz.

8.3 printf ile Matris Yazdırmak: `%d` ve `%3d`

Soru: Matrisi yazdırırken `%d` ile yazdırmıyor, `%3d` ile yazdırıyor. Oysa üç karakter yok. Neden?

Cevap: Bu formatlı bir yazım şeklidir. Oraya 3 koyarsanız 3 basamak varmış gibi düşünerek yazar. 3 basamak yoksa ikisini boş bırakır, birine olan basamağı yazar. `%d` kullanınca da yazdırır, yalnızca sayılar birbirine yapışık görünür: 1, 2, 3 için 123 gibi.

Python ile aynı biçim denendi: `%3d` ile 1, 2, 3 için her sayı 3 karakterlik alana sağa yaslı yazılıyor, `%d` ile 123 çıkıyor.

Soru: Ben `%d` yazınca rastgele değerler görüyorum, `%3d` düzeltiyor.

Cevap: Böyle olmaması gerekir. Bir öğrencinin belirttiği gibi atama yapılmamış olabilir. Atama yapmadan matrisi yazdırıyorsanız rastgele değer görmeniz normaldir. Kesin yorum için kodu görmek gerekir.

8.4 Farklı Hocalardan Öğrenmek

Bir öğrencinin C pratiğiyle ilgili yorumu üzerine hoca şunu söyledi: Her hocanın kendi stili vardır. Anlatılan konu ve dersin içeriği değişmese de her hocanın yorumlayış ve aktarış biçimi farklıdır. Bu yüzden mümkün olduğunca farklı hocalardan ders alarak farklı stilleri görmek uzun vadede fayda sağlar. Kiminin stili size uyar, kimininki uymaz. Ama hepsini görmek gerekir.

Öğrenci bunu yemek benzetmesiyle anlattı: Bir yemeğin tuzlu olduğunu söyleyebilmek için önce tadına bakmak gerekir. Hoca da benzetmeyi sürdürdü: Bir sebzeyi ya da meyveyi yemiyorum demeden önce tadına bakmak, hatta farklı zamanlarda bakmak gerekir. Gençken alınan tat ile yaş ilerleyince alınan tat farklı olabilir. Bilgilerinizi tazelerken de bunu göz önünde bulundurun.

> **Hocanın vurgusu:** Mümkün olduğunca farklı hocalardan ders alın ve farklı stilleri görün.

8.5 Çekirdek (Kernel) ile Merkezi İşlem Birimi Farkı

Soru: Geçen derste işletim sisteminde kullanıcı arayüzü ve çekirdek (kernel) vardı. Çekirdek merkezi işlem biriminin görevini görüyor gibi. Arada fark var mı?

Cevap: Çekirdeği CPU (central processing unit, merkezi işlemci) gibi düşünmeyin. Çekirdek yalnızca CPU'dan sorumlu değildir. Bellekten ve giriş/çıkış işlemlerinden de sorumludur, yani bilgisayarın bütün parçalarının temel işlerinden sorumludur. İşletim sisteminin diğer kısımları daha çok kullanıcı ile etkileşimde olunan kısımlardır.

Soru: Çekirdek yazılımsal, merkezi işlem birimi donanımsal bir şey değil miydi?

Cevap: Evet. Çekirdek yazılımdır ve işletim sisteminin merkezi (core) noktasıdır. Merkezi işlem birimi ise tamamen donanımsal bir yapıdır.

Dersin Toparlaması

Bu derste aynı düşünme alışkanlığı farklı problemlerle çalışıldı. Sütun silmede uç durumların ayrı ayrı denenmesi gerektiği görüldü. Kümelerde simetri kontrolü için önce bağıntı matrisi, sonra iki sütunlu listeyle `flag` kullanan çözüm kuruldu. Matrisli çözümde transpozun yarısını gezmenin yeterli olduğu, ilk eşitsizlikte çıkılacağı için `while` gerektiği, büyük sayılarda ise matrisin çok yer kaplayacağı konuşuldu. Polinom örneğinde de girdinin biçimi değiştikçe problemin nasıl değiştiği görüldü, string girdide rakamın değeri `'0'` farkıyla elde edildi.

Hoca bir sonraki iki hafta uygulama saatinde kod yazacağını söyledi. Böylece ikinci dönemdeki yapısal programlama dersine giderken C görerek ve hissederek gidilecektir. Kapanışta farklı problemlere bakmayı, soru üretmeyi ve bunları çözmeyi önerdi. Hocaya göre bunu yapanlar için ikinci ve üçüncü sınıftaki algoritma dersleri çok daha kolay ve keyifli geçecektir.

> **Hocanın vurgusu:** Hem soru sorun hem fikir üretin. Sadece sorarak bir yere varamazsınız, sadece fikir üreterek de bir yere varamazsınız. Fikrinizin doğruluğu ve yanlışlığı konuşuldukça gelişirsiniz.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.